

DAT12-3

Pandas I

[Module page](#)

Version of 13 September 2026

About this textbook

This year, for the first time, every Bachelor course at Albert School has its own textbook, thanks to your teachers and AI.

- The textbook is a help for working on the course at home, after the class.
- The course as taught in class remains the reference, and prevails over the textbook wherever the two differ.
- This is a first version. Help us improve it by sharing your feedback with your teacher.

Contents

Preface	4
1. DataFrames and Series	6
1.1. Two objects, one built from the other	6
1.2. Loading data from a file	7
1.3. Inspecting before you touch	7
2. Selection and Filtering	11
2.1. Selecting columns	11
2.2. Selecting rows: loc and iloc	11
2.3. Filtering with boolean indexing	13
3. Missing Values	15
3.1. Step 1 — Find them	15
3.2. Step 2 — Decide: drop or impute	15
4. Groupby and Aggregation	17
4.1. Aggregating several ways at once	17
4.2. Grouping by several dimensions	18
5. Merging and Concatenating	20
5.1. Concatenation: stacking like with like	20
5.2. Merging: joining on a key	21
6. Reshaping: Wide and Long	24
7. Applying Functions	26
7.1. Series apply and DataFrame apply	26
8. The End-to-End Pipeline	28
9. Visualization	30
9.1. Choosing the right chart	30
9.2. Bar chart: comparing across categories	30
9.3. Histogram: the shape of one variable	31
9.4. Line chart: a value along an ordered axis	32
9.5. Scatter plot: relating two variables	33
10. Descriptive Reporting	36

Preface

In your Python course you learned to read a CSV file the hard way. You opened it with a context manager, looped over its lines, split each line on a comma, and built up lists or dictionaries by hand. That works, and it taught you what a table really is: rows, columns, and the constant effort of keeping them lined up. But it does not scale. As soon as you want the average of a column, the rows where one value is above another, or a summary broken down by category, the hand-written loops pile up. Each loop is a new place for a mistake that no one will notice.

This course gives you the standard professional tool for that work: **pandas**. Pandas does not give you a new kind of data. A table is still a table, the same one you built by hand before. What it gives you is a **vocabulary** for working on whole tables at once: select these rows, group by that column, join this table to that one, change a wide table into a long one. Where you once wrote a ten-line loop, you now write one line that says what you want, and pandas runs the loop for you, correctly, on a million rows as easily as on ten.

The skill this course builds is not memorising method names. It is learning to **read a data question as a sequence of table operations**, and then to read the answer back as a sentence a person can act on. By the end you will take a raw CSV file, inspect it, clean it, summarise it, and write a short paragraph that says what it means. That whole path, from a messy file to a result you can defend, is the goal of the course, and every chapter is one step along it.

A note on running the code. Pandas is a third-party library. Install it into your virtual environment with `pip install pandas` (add `openpyxl` if you read Excel files, and `matplotlib` and `seaborn` for the charts in Chapter 9), exactly as you learned to install any package. Then, by universal convention, import it under the short name `pd`:

```
import pandas as pd
```

Every code example below assumes this line has already run.

One dataset runs through the whole book. It is a small table of shops in a retail chain, one row per shop. Save the following text as a file called `shops.csv` so you can run every example and exercise yourself:

```
shop,region,category,revenue,orders,rating
Aster,North,Books,1200,150,4.5
Birch,North,Toys,900,120,4.1
Cedar,South,Books,1500,210,4.7
Dahlia,South,Food,600,300,3.9
Elm,East,Toys,1100,140,
Fern,East,Food,-50,80,4.0
Gorse,West,Books,2000,260,4.8
Holly,West,Toys,750,95,3.6
Iris,North,Food,480,220,4.2
Juniper,South,Toys,1300,175,
Larch,East,Books,1700,230,4.6
Maple,,Food,540,240,3.8
Olive,West,Food,820,280,4.3
Pine,North,Books,1600,205,4.4
```

Each row records a shop's region, product category, monthly revenue (in a single shared currency), number of orders, and average customer rating on a scale from 1 to 5. The table is deliberately imperfect: a few values are missing, and one is impossible. Learning to notice and handle such flaws is a large part of this course.

1. DataFrames and Series

1.1. Two objects, one built from the other

Everything in pandas is built from two objects. Meet the smaller one first.

Definition 1 (Series). A **Series** is a one-dimensional array of values together with an **index**: a sequence of labels, one per value. It is, in effect, a single column whose rows have names.

Definition 2 (DataFrame). A **DataFrame** is a two-dimensional table: a set of Series that share one common index (the row labels) and are told apart by column names. Selecting one column of a DataFrame gives you back a Series.

The link between the two is the key idea, so let us make it concrete. Here are the first three rows of `shops.csv`:

	shop	region	revenue
0	Aster	North	1200
1	Birch	North	900
2	Cedar	South	1500

The whole table is a DataFrame. Its leftmost column, 0, 1, 2 in grey, is not data. It is the **index**, the shared row labels. The revenue column on its own, carrying that same index, is a Series:

```
0    1200
1     900
2    1500
Name: revenue, dtype: int64
```

Read the DataFrame as three Series (shop, region, revenue) standing side by side, all lined up on the same index. That mental picture explains every operation that follows.

	shop	region	revenue
0	Aster	North	1200
1	Birch	North	900
2	Cedar	South	1500

the revenue column, alone, is a Series

Figure 1: A DataFrame is several Series that share one index. The grey index column holds the row labels; the highlighted revenue column, carried with that index, is itself a Series.

1.2. Loading data from a file

You rarely type a table by hand; you load it from a file. Pandas gives you one reader per common format, and each one returns a DataFrame:

```
df = pd.read_csv("shops.csv")
df = pd.read_excel("shops.xlsx")    # needs the openpyxl package
df = pd.read_json("shops.json")
```

`read_csv` is the one you will use most, and it has many options. The two you will reach for first handle the most common surprises:

```
df = pd.read_csv("shops.csv", sep=";")          # some CSVs use ; instead of ,
df = pd.read_csv("shops.csv", encoding="latin-1") # if accented text looks wrong
```

Excel and JSON files bring one extra question each. An Excel workbook can hold several sheets, so `read_excel` reads the first sheet unless you name another with `sheet_name="Sales"`. A JSON file can be laid out in more than one way; if `read_json` does not produce the table you expect, the `orient` option (for example `orient="records"`) tells pandas how the data is arranged.

Example — The same table, three ways. You already have `shops.csv`. To watch the Excel and JSON readers work without hunting down other files, write that same table out to the two other formats and read each back:

```
df = pd.read_csv("shops.csv")
df.to_json("shops.json", orient="records")    # write a JSON copy
df.to_excel("shops.xlsx", index=False)        # write an Excel copy (needs openpyxl)

pd.read_json("shops.json").shape              # (14, 6)
pd.read_excel("shops.xlsx").shape             # (14, 6)
```

All three readers return the same (14, 6) DataFrame. One difference is worth noting: `read_json` brings the numbers back **as** numbers (revenue as `int64`, rating as `float64`), because JSON keeps numbers and text apart, whereas a CSV, as the next box explains, is nothing but text.

Pitfall. A CSV file carries no type information; it is just text. Pandas **guesses** each column's type from its contents, and a single unexpected value can spoil the guess. One - or the word missing typed into a column of numbers makes pandas read the **whole** column as text, and every later sum or average on it will fail or give a wrong answer. (Pandas does recognise the common markers NA, N/A, and null as missing values, so those **do not** turn the column into text; other stray values do.) The cure is simple: check the types right after loading, before you compute anything. The next section shows how.

1.3. Inspecting before you touch

The first thing to do with any freshly loaded DataFrame is **look at it**, not compute with it. Five tools answer the five questions you should always ask.

Definition 3 (The five inspection tools).

- `df.shape` — a pair (rows, columns): how big is the table?
- `df.head(n)` — the first `n` rows (5 by default): what do the values look like?
- `df.dtypes` — the stored type of each column: is each column the type I expect?
- `df.info()` — column names, non-null counts, and types together: where are values missing?
- `df.describe()` — count, mean, standard deviation, minimum, the three quartiles, and maximum of each numeric column: how is each number spread out?

Example — Looking at the shops table. Load the file, then look before you compute. Start with its size and its first rows:

```
df = pd.read_csv("shops.csv")
df.shape          # (14, 6): 14 rows, 6 columns
df.head()
```

	shop	region	category	revenue	orders	rating
0	Aster	North	Books	1200	150	4.5
1	Birch	North	Toys	900	120	4.1
2	Cedar	South	Books	1500	210	4.7
3	Dahlia	South	Food	600	300	3.9
4	Elm	East	Toys	1100	140	NaN

Five rows, six columns, and already something to notice: Elm's rating shows as NaN, a missing value (Chapter 3). Now ask the column types:

```
df.dtypes
```

```
shop          str
region        str
category      str
revenue       int64
orders        int64
rating        float64
dtype: object
```

Two things stand out. The text columns (shop, region, category) have a text type, and the number columns have a numeric type (int64 for whole numbers, float64 for numbers with a decimal point). If revenue had shown up as text here, you would know at once that some non-number had slipped into it.

`df.info()` adds the missing-value picture:

```
<class 'pandas.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -

```

```

0   shop      14 non-null    str
1   region    13 non-null    str
2   category  14 non-null    str
3   revenue   14 non-null    int64
4   orders    14 non-null    int64
5   rating    12 non-null    float64
dtypes: float64(1), int64(2), str(3)
memory usage: 804.0 bytes

```

Every column should have 14 non-null values, one per row. `region` has only 13 and `rating` only 12, so one region and two ratings are missing. You have found the gaps before doing any analysis.

Remark. In pandas version 3.0 and later, text columns show the type `str`, as above. In earlier versions the same columns show `object`. Both mean the same thing for us: “this column is not a number type.”

`describe` is the richest of the five, but it uses several statistical words. Here is what each one means, in case they are new to you.

Definition 4 (Reading a `describe` summary). For each numeric column, `describe` reports:

- **count** — how many values are present (missing values are not counted).
- **mean** — the average: add all the values and divide by the count.
- **std** (standard deviation) — how spread out the values are. Roughly, it is the typical distance of a value from the mean. A small **std** means the values sit close together; a large **std** means they are far apart.
- **min** and **max** — the smallest and largest values.
- **25%, 50%, 75%** (the quartiles) — sort the values from low to high. The **50%** value, the **median**, is the one in the middle: half the values are below it. A quarter of the values are below the **25%** value, and three quarters are below the **75%** value. Together they show where the middle half of the data sits.

Example — Reading `describe` out loud. Calling `df.describe()` on the `shops` table prints, for `revenue`:

```

count      revenue      orders      rating
mean    1031.428571    193.214286    4.241667
std       559.915614     68.461307    0.375278
min       -50.000000     80.000000    3.600000
25%       637.500000    142.500000    3.975000
50%      1000.000000    207.500000    4.250000
75%      1450.000000    237.500000    4.525000
max      2000.000000    300.000000    4.800000

```

Read it as sentences, not as figures. “There are 14 revenue values (none missing), but only 12 ratings, so two ratings are gone. A typical shop makes about 1000 in revenue (the median, the 50% row), and the middle half of shops sit between 638 and 1450. The ratings are tightly grouped: a

standard deviation of only 0.38 around a mean of 4.24 means almost every shop is rated close to 4.2. And **something is wrong**: a minimum revenue of -50 is impossible, so that row holds an error, not a real figure.” The numbers did the looking; you did the interpreting. That last step is the real skill.

Pitfall. `df.describe()` quietly ignores every non-numeric column and every missing value. A tidy-looking summary can hide the fact that some rows have no value at all: the count row is what reveals it. Always compare each column’s count against the row total from `df.shape`; a gap is missing data you have not dealt with yet.

Remark. `head` shows the **top** of the file, which is often its most regular part. Pair it with `df.tail()` (the last rows) and, once you know `describe`, with a glance at the minimum and maximum. Errors often hide in the places you are not looking.

Exercises

1.1. Load `shops.csv` into a DataFrame. Report its shape, its dtypes, and the output of `describe`. Name one column whose count in `describe` is below the row total, and say how many of its values are missing.

1.2. Write the loaded table out to JSON with `df.to_json("shops.json", orient="records")`, read it back with `read_json`, and check that the result has the same shape and the same dtypes as the original.

Answers. (1) `df.shape` is (14, 6). `describe` shows count 12 for rating (2 missing) and 14 for revenue and orders; region, being text, does not appear in `describe`, but `info` shows it has 13 non-null values (1 missing).

(2) `back = pd.read_json("shops.json")` gives back `back.shape` equal to (14, 6) and `back.dtypes` equal to the original’s, including revenue as `int64` and rating as `float64` — JSON records that these columns hold numbers, so no re-guessing is needed.

2. Selection and Filtering

Loading gives you the whole table; analysis almost always needs only a **part** of it: some columns, some rows, or the rows that meet a condition. Pandas keeps these three jobs separate, and the most common beginner confusion comes from mixing up the tools, so we keep them clearly apart.

2.1. Selecting columns

A single column comes out as a Series; a list of columns comes out as a smaller DataFrame:

```
df["revenue"]          # one column  -> Series
df[["region", "revenue"]] # several columns -> DataFrame
```

Pitfall. The double brackets are not a typo. `df["revenue"]` (single brackets) returns a Series; `df[["revenue"]]` (a one-element **list** of names) returns a one-column DataFrame. They print almost the same but behave differently afterwards, because a Series and a DataFrame support different methods. When you mean “a table of these columns”, use a list, even if it has only one element.

2.2. Selecting rows: loc and iloc

To pick **rows**, pandas gives you two tools that look alike but mean opposite things. Getting them straight now will save you time later.

Definition 5 (loc and iloc).

- `df.loc[rows, cols]` selects by **label**: the index labels and the column names.
- `df.iloc[rows, cols]` selects by **integer position**: the row and column numbers, counting from 0, whatever the labels are.

Each takes a row selector and an optional column selector, separated by a comma.

When the index is the default 0, 1, 2, ..., the label and the position of a row are the same number, so `loc` and `iloc` seem interchangeable. They stop being interchangeable the moment the labels are not those positions. You change the labels with `set_index`:

```
df2 = df.set_index("shop")    # the shop names become the row labels
```

Example — The same value, reached two ways. In `df2`, the row labels are now shop names, and revenue is the third data column (position 2, counting from 0).

```
df2.loc["Cedar", "revenue"]  # by label    -> 1500
df2.iloc[2, 2]               # by position -> 1500
```

Both give 1500 only because Cedar happens to sit in row position 2. Now sort the table by revenue, which reorders the rows:

```
df3 = df2.sort_values("revenue")
df3.loc["Cedar", "revenue"]    # still 1500 – loc follows the label
df3.iloc[2, 2]                 # now 540 – iloc follows the position
```

`loc["Cedar"]` still finds Cedar, because Cedar's label did not change. `iloc[2]` now finds whichever shop moved into position 2. **Use `loc` when you know the label, and `iloc` when you know the position.**

		pos 0	pos 1	pos 2	pos 3	pos 4	pos 5
		shop	region	category	revenue	orders	rating
pos 0	0	Aster	North	Books	1200	150	4.5
pos 1	1	Birch	North	Toys	900	120	4.1
pos 2	2	Cedar	South	Books	1500	210	4.7

`loc[1, "revenue"]` and `iloc[1, 3]`
both name the shaded cell: **900**

Figure 2: `loc` addresses by **label** — row label 1 and column name revenue; `iloc` addresses by **0-based position** — the violet “pos” guides, row 1 and column 3. With the default index the label equals the position on each axis, so `loc[1, "revenue"]` and `iloc[1, 3]` name the very same shaded cell: Birch's revenue, 900.

Both `loc` and `iloc` take **slices**, exactly as a Python list does, and a slice is how you pull out a whole **range** of rows or columns in one step instead of naming each. Give a slice on each side of the comma: one for the rows, one for the columns.

Example — Slicing a block of rows and columns. Take the block of rows 2 through 4 and, within them, the columns from shop to category:

```
df.loc[2:4, "shop":"category"]
```

```
   shop region category
2  Cedar  South    Books
3  Dahlia South    Food
4    Elm   East    Toys
```

Both selectors are slices. `2:4` picks the rows; `"shop":"category"` picks every column from shop up to category. A bare `:` means “keep all” on that side, so `df.loc[2:4, :]` is those three rows with every column, and `df.iloc[:, 0:3]` is every row with the first three columns. The position form is the same idea with numbers: `df.iloc[2:5, 0:3]` reaches the very same block.

Pitfall. With `loc`, a slice **includes** its right end: `df.loc[0:5]` returns the rows labelled 0 through 5, which is six rows. With `iloc`, as everywhere else in Python, a slice **excludes** it: `df.iloc[0:5]`

returns positions 0 through 4, which is five rows. This difference of one row is a very common pandas bug. The rule to remember: labels are inclusive, positions are exclusive.

2.3. Filtering with boolean indexing

The most useful kind of selection is by **condition**: keep the rows where something is true. It works in two steps, and seeing them one at a time makes it clear.

Example — A condition is itself a column. Write a comparison on a column, and pandas checks it for every row. The result is a **boolean Series**: a column of True and False values.

```
df["rating"] > 4.3
```

```
0    True
1   False
2    True
Name: rating, dtype: bool
```

Hand that boolean Series back to the DataFrame in brackets, and pandas keeps only the rows marked True:

```
df[df["rating"] > 4.3]    # only the shops rated above 4.3
```

So filtering is not a special command. It is computing a column of True/False values and using it as a mask. Once you see it this way, combining conditions is straightforward.

Definition 6 (Combining conditions). Join boolean Series with & (and), | (or), and ~ (not). **Each condition must be wrapped in its own parentheses:**

```
df[(df["rating"] > 4.3) & (df["region"] == "North")]
```

Pitfall. Two traps hide in that one line. First, you must use & and |, **not** the Python keywords and and or. The keywords work on single True/False values, not on whole columns, and raise an error here. Second, & and | bind more tightly than > and ==, so without the inner parentheses the expression `df["rating"] > 4.3 & df["region"] == "North"` is grouped wrongly and fails. Always put parentheses around each comparison.

Exercises

2.1. Select the revenue column as a Series, and then select revenue as a one-column DataFrame. Explain the difference between the two results.

2.2. Set `shop` as the index with `set_index`. Retrieve Cedar's revenue with `loc`, and the value in row position 2, column position 2 with `iloc`. Then sort the table by revenue and show that `iloc[2, 2]` now returns a different value while the `loc` lookup does not.

2.3. Using a single `loc` call with a slice on each side of the comma, select the block of rows 2 through 5 and the columns from `shop` to `revenue`. Say how many rows it returns, and explain why a slice of row **labels** behaves differently at its right end from a slice of row **positions**.

2.4. Select all rows where revenue is above its own mean **and** region equals "North", using a single boolean expression with correct parentheses.

Answers. (1) `df["revenue"]` is a Series (one-dimensional, no column header). `df[["revenue"]]` is a DataFrame with one column. They print almost the same, but only the DataFrame supports DataFrame methods; a Series and a DataFrame are different types.

(2) `df2 = df.set_index("shop"); df2.loc["Cedar", "revenue"]` is 1500, and `df2.iloc[2, 2]` is also 1500 because Cedar is in position 2. After `df3 = df2.sort_values("revenue")`, `df3.iloc[2, 2]` is 540 (Maple, now in position 2), while `df3.loc["Cedar", "revenue"]` is still 1500. `loc` follows the label; `iloc` follows the position.

(3) `df.loc[2:5, "shop":"revenue"]` returns **four** rows (labels 2, 3, 4, 5), because a `loc` label slice includes its right end. The position equivalent `df.iloc[2:6, 0:4]` excludes its right end, so it needs 6 (not 5) to reach the same four rows. Labels are inclusive; positions are exclusive.

(4) `df[(df["revenue"] > df["revenue"].mean()) & (df["region"] == "North")]`. The mean revenue is about 1031, so this keeps the two North shops above it, Aster (1200) and Pine (1600), and drops Birch (900) and Iris (480).

3. Missing Values

Real data has holes. A customer skipped a field, a sensor missed a reading, a join found no match. Pandas marks every such hole with the special value NaN (“Not a Number”). A large part of professional data work is deciding, column by column, what to do about these holes. The decision is yours to **justify**; pandas only gives you the tools.

3.1. Step 1 — Find them

You cannot handle what you cannot see, so find the gaps first.

```
df.isna()           # a boolean DataFrame: True wherever a value is missing
df.isna().sum()     # number missing per column (each True counts as 1)
df.isna().mean()    # fraction missing per column
```

Example — Reading the missing-value profile. On the shops table, `df.isna().sum()` returns:

```
shop      0
region    1
category  0
revenue    0
orders    0
rating     2
dtype: int64
```

Now you know the shape of the problem before touching it. revenue, orders, shop, and category are complete; region is missing for one shop; rating is missing for two. Different columns will need different decisions, and this profile is what tells you so.

3.2. Step 2 — Decide: drop or impute

Once you have found a missing value, it meets one of two fates: the row (or column) is **dropped**, or the gap is **filled**, which is called **imputing**.

```
df.dropna()         # drop every row that has any NaN
df.dropna(subset=["region"]) # drop only rows missing a region
df.dropna(axis=1)    # drop every column that has any NaN
```

```
df["rating"] = df["rating"].fillna(df["rating"].median()) # numeric -> median or mean
df["region"] = df["region"].fillna("Unknown")             # text -> a constant or the
mode
```

Definition 7 (Choosing a strategy).

- **Drop rows** when the missing values are few and scattered, so that losing those rows does not bias the result.

- **Drop a column** when it is missing so often that it carries little information.
- **Impute a numeric column** with its **mean** when the values are spread roughly evenly around the centre, or with its **median** when they are lopsided or contain outliers (the median is not pulled around by extreme values).
- **Impute a text column** with a constant such as "Unknown", or with the **mode** (the most frequent value) when you want a best guess.

Whichever you choose, **state it and say why**. The choice changes the result, so it must be one you can defend.

Pitfall. Imputing is not free. Filling the one missing region with "Unknown" creates a new category that will then appear in any `groupby("region")`; filling missing ratings with the mean makes the ratings look less spread out than they really are, which lowers the standard deviation. There is no neutral choice: dropping throws away real rows, and imputing invents values. Pick the smaller distortion **for the question you are answering**, and write down what you did.

Pitfall. `df.dropna()` and `df.fillna()` return a **new** DataFrame and leave the original unchanged. Writing `df.dropna()` on a line by itself does nothing lasting: you must reassign the result, as in `df = df.dropna()`. Forgetting the assignment, then wondering why the NaN values are still there, is a very common mistake.

Exercises

3.1. Count the missing values in each column. Then impute the rating column with its median and the region column with the constant "Unknown", and write one sentence justifying each choice.

Answers. (1) `df.isna().sum()` gives region 1 and rating 2, others 0. `df["rating"] = df["rating"].fillna(df["rating"].median()); df["region"] = df["region"].fillna("Unknown")`. The median suits rating because it is a numeric column and the median is not distorted by outliers; a constant suits region because there is no meaningful average of a text label, and "Unknown" keeps the row while marking it as a gap.

4. Groupby and Aggregation

So far every operation has returned rows. The real step up in analysis is **summarising**: turning many rows into one number per category, such as average revenue **per region** or total revenue **per month**. Pandas does this with groupby, and the pattern behind it is worth naming, because you will use it constantly.

Definition 8 (Split-apply-combine). groupby follows three steps. **Split** the rows into groups by the value of one or more columns. **Apply** an aggregation (mean, sum, count, and so on) to a chosen column within each group, reducing it to one value. **Combine** those per-group values into a new, smaller table indexed by the group.

Example — Average revenue per region.

```
df.groupby("region")["revenue"].mean()
```

Read from left to right, this says: split the rows by region, take the revenue column of each group, and average it. The result is one number per region:

```
region
East      916.666667
North    1045.000000
South     1133.333333
West     1190.000000
Name: revenue, dtype: float64
```

Fourteen rows became four, and the four answer the question “where do we earn the most on average?” Turning rows into a short answer like this is what groupby is for.

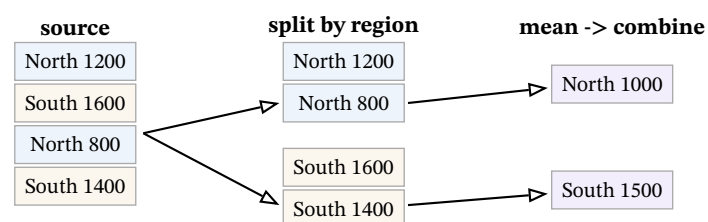


Figure 3: Split-apply-combine on a small example: rows are split into groups by region, the mean of revenue is applied within each group, and the per-group results are combined into one row per region.

4.1. Aggregating several ways at once

agg lets one call compute different aggregations on different columns, or several on one column:

```
df.groupby("region").agg({"revenue": "sum", "rating": "mean"}) # sum revenue, mean
rating
df.groupby("region")["revenue"].agg(["sum", "mean", "count"]) # three at once
```

The second call gives, per region, the total revenue, the average revenue, and the number of shops:

	sum	mean	count
region			
East	2750	916.666667	3
North	4180	1045.000000	4
South	3400	1133.333333	3
West	3570	1190.000000	3

4.2. Grouping by several dimensions

Pass a **list** of columns to summarise along more than one dimension at once:

```
df.groupby(["region", "category"])["revenue"].sum()
```

region	category	
East	Books	1700
	Food	-50
	Toys	1100
North	Books	2800
	Food	480
	Toys	900
South	Books	1500
	Food	600
	Toys	1300
West	Books	2000
	Food	820
	Toys	750

Name: revenue, dtype: int64

This gives one total per (region, category) pair, the table answer to “how does revenue split across categories within each region?” Notice the result’s left edge: it now has **two** levels. The outer level (East, North, ...) is printed once and the inner level (Books, Food, Toys) is listed beneath it, like an indented outline. This is a **MultiIndex**: each value is labelled not by one key but by the pair (region, category). Reading down the first block, East’s shops made 1700 on Books, lost 50 on Food, and made 1100 on Toys.

Pitfall. By default groupby returns a table **indexed by the grouping columns** — here the two-level (region, category) index above — not ordinary columns you can select with `df["region"]`. To get a flat DataFrame back, which most later steps expect, add `.reset_index()`, which turns each index level into a normal column:

```
df.groupby(["region", "category"])["revenue"].sum().reset_index()
```

	region	category	revenue
0	East	Books	1700
1	East	Food	-50

2	East	Toys	1100
3	North	Books	2800
4	North	Food	480
5	North	Toys	900
6	South	Books	1500
7	South	Food	600
8	South	Toys	1300
9	West	Books	2000
10	West	Food	820
11	West	Toys	750

The same twelve totals are now three plain columns — region, category, revenue — with a fresh 0...11 index you can filter and merge like any other table.

Remark. `groupby` skips NaN group keys completely: the shop whose region is missing simply disappears from a `groupby("region")` summary. If those rows matter, handle the missing values (Chapter 3) **before** you group, or they will be dropped from your totals without warning.

Exercises

4.1. Group the shops by region and compute the sum, mean, and count of revenue in one call. Then group by region **and** category together, and describe what a single row of the result represents.

Answers. (1) `df.groupby("region")["revenue"].agg(["sum", "mean", "count"])` gives one row per region. `df.groupby(["region", "category"])["revenue"].sum()` gives one row per (region, category) pair; a single row is the total revenue of one category within one region, for example Books in North.

5. Merging and Concatenating

Data rarely arrives in one file. Shops sit in one table, the manager of each region in another; this year's sales sit beside last year's. Combining tables comes in two kinds, and confusing them is a classic error, so let us name the difference up front. **Concatenation** stacks tables that have the same shape. **Merging** joins tables that share a key.

5.1. Concatenation: stacking like with like

Concatenation glues whole tables together. It goes in one of two directions: stack **rows** (set one table's rows under another's) or stack **columns** (set one table's columns beside another's).

```
pd.concat([df1, df2])           # stack rows: more observations, same columns
pd.concat([df1, df2], axis=1)   # stack columns: more variables, same rows
```

Use `concat` when the pieces are **the same kind of thing**, such as January's shops under February's shops, all with identical columns.

Example — Stacking two months of shops. Suppose January's shops and February's shops arrived in two separate tables with the same columns:

```
jan = pd.DataFrame({"shop": ["Aster", "Birch"], "revenue": [1200, 900]})
feb = pd.DataFrame({"shop": ["Cedar", "Dahlia"], "revenue": [1500, 600]})
pd.concat([jan, feb])
```

`concat` sets the second table directly under the first, lining the columns up by name:

	shop	revenue
0	Aster	1200
1	Birch	900
0	Cedar	1500
1	Dahlia	600

Four rows and the same two columns — but look at the index down the left edge: 0,

1, 0, 1. Each table kept its own row labels, so they now repeat. That is exactly the trap the next box is about.

Pitfall. When you stack rows, `concat` keeps each piece's original index, so the result can carry repeated labels (0, 1, 0, 1 in the example above). Repeated labels break a later `loc` lookup, which no longer points to a single row. Pass `ignore_index=True` to throw the old labels away and number the combined rows fresh from 0:

```
pd.concat([jan, feb], ignore_index=True) # the index becomes 0, 1, 2, 3
```

Stacking **columns** with `axis=1` sets one table's columns beside another's, but it lines the rows up by **index label**, not by position. That surprises people, so see it once.

Example — Stacking columns lines up on the index. Two frames whose row labels do not fully match:

```
a = pd.DataFrame({"shop": ["Aster", "Birch"]}, index=[0, 1])
b = pd.DataFrame({"manager": ["Ada", "Ben"]}, index=[1, 2])
pd.concat([a, b], axis=1)
```

	shop	manager
0	Aster	NaN
1	Birch	Ada
2	NaN	Ben

Only label 1 is present in both frames, so only that row lines up (Birch with Ada). Labels 0 and 2 exist on one side alone, and pandas fills the other side with NaN. To set rows beside each other **by position** — the thing people usually mean — make sure both frames carry the same index first, for example by calling `reset_index(drop=True)` on each.

5.2. Merging: joining on a key

Example — Attaching a region's manager to each shop. A second table lists the manager of each region:

```
regions = pd.DataFrame({
    "region": ["North", "South", "East", "Central"],
    "manager": ["Ada", "Ben", "Cira", "Dan"],
})
```

To put the right manager beside each shop, **merge** the two tables on their shared key, region:

```
pd.merge(df, regions, on="region", how="left")
```

Pandas looks up each shop's region in the regions table and copies the matching manager across. One key value can match many rows: every North shop receives the manager “Ada”.

The `how` argument decides **which rows survive when a key is missing on one side**. This is the single most important and most misunderstood choice in this chapter.

Definition 9 (The four join types). Merging a left table and a right table on a key:

- **inner** — keep only keys present in **both** tables. No unmatched rows survive.
- **left** — keep **every** key of left; where right has no match, fill its columns with NaN.
- **right** — keep every key of right; the mirror image of left.
- **outer** — keep every key of **either** table; fill both sides' gaps with NaN.

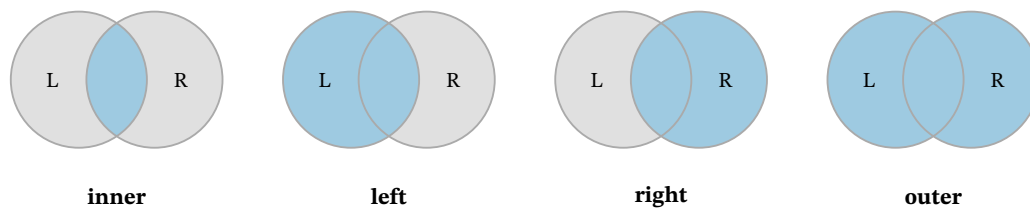


Figure 4: The four joins differ only in which keys they keep. Blue marks the rows a join keeps; grey marks the rows it drops. Inner keeps only the matched overlap; left keeps all of L (overlap included); right keeps all of R; outer keeps everything.

Example — How the join type changes the answer. The regions table lists North, South, East, and Central. The shops table has North, South, East, and West, and one shop with no region at all. The four join types keep four different sets of rows, and the row count alone already shows it:

```
len(pd.merge(df, regions, on="region", how="inner")) # 10
len(pd.merge(df, regions, on="region", how="left"))  # 14
len(pd.merge(df, regions, on="region", how="right")) # 11
len(pd.merge(df, regions, on="region", how="outer")) # 15
```

- **inner** keeps only the 10 shops whose region (North, South, or East) is also in regions. The 3 West shops and the 1 shop with a missing region are dropped, because “West” and a missing value match nothing.
- **left** keeps all 14 shops and fills manager with NaN for those same 4.
- **right** keeps every region in regions, so it adds “Central” — which has a manager but no shops — as one row whose shop columns are NaN. That is 11 rows: the 10 matched shops plus Central.
- **outer** keeps everything — all 14 shops **and** Central — for 15 rows.

The outer join is the one to read closely, because it is the only one that fills NaN on **both** sides at once:

```
pd.merge(df, regions, on="region", how="outer")[["shop", "region", "manager"]]
```

	shop	region	manager
0	NaN	Central	Dan
1	Elm	East	Cira
2	Fern	East	Cira
3	Larch	East	Cira
4	Aster	North	Ada
5	Birch	North	Ada
6	Iris	North	Ada
7	Pine	North	Ada
8	Cedar	South	Ben
9	Dahlia	South	Ben
10	Juniper	South	Ben
11	Gorse	West	NaN
12	Holly	West	NaN
13	Olive	West	NaN
14	Maple	NaN	NaN

Read the two kinds of NaN. Central (row 0) is a key in regions with no shop, so its **shop** columns are empty — that is what a right join keeps and a left join drops. The West shops and the region-less Maple (rows 11–14) are keys in the shops table with no manager, so their **manager** column is empty — that is what a left join keeps and a right join drops. An inner join would have dropped all five of these rows; the outer join keeps them all. If your question is “total revenue by manager”, the inner join silently ignores 4 real shops, and only a right or outer join ever shows you that Central has no shops at all. The join type decides which rows your analysis is even aware of.

Pitfall. After a merge, always check the row count. An inner join that quietly halves your rows means most keys did not match, which usually points to a dirty key (“FR” against “fr”, or a number stored as text against a real number) rather than genuinely absent data. A merge that **grows** the row count beyond either input means the key was not unique on one side, so rows were multiplied. Compare `len(result)` against what you expected, every time.

Exercises

5.1. Merge `df` with the `regions` table from this chapter with each of the four join types (inner, left, right, outer). Report the row count of each, and explain, in terms of unmatched keys, why they differ. Then, in the outer result, point to one row whose manager is NaN and one whose shop is NaN, and say what each of those two NaNs means.

5.2. Build two small DataFrames with the same columns (two shops each, say), stack them with `concat`, and show that the row labels repeat. Then stack them again with `ignore_index=True` and confirm the labels now run 0, 1, 2, 3.

5.3. Build two one-column DataFrames whose indices only partly overlap (for instance labels [0, 1] and [1, 2]), stack them side by side with `pd.concat(..., axis=1)`, and explain where each NaN in the result comes from.

Answers. (1) inner gives 10 rows, left 14, right 11, outer 15. The inner join drops the 3 West shops and the 1 shop with a missing region, because “West” and a missing value have no matching key in `regions`. The left join keeps all 14 shops and fills `manager` with NaN for those 4. The right join keeps every key of `regions`, adding “Central” (a manager with no shops) as one row with a NaN shop: 10 matched shops + Central = 11. The outer join keeps every key of either side: all 14 shops + Central = 15. In

the outer result, Central’s row has manager “Dan” but NaN shop (a region with no shop), while each West shop and Maple has a shop but NaN manager (a shop whose region is not in `regions`).

(2) With `jan` and `feb` as in the chapter example, `pd.concat([jan, feb])` has the index 0, 1, 0, 1 (each piece’s own labels, repeated), while `pd.concat([jan, feb], ignore_index=True)` has the index 0, 1, 2, 3. The repeated labels would break a later `loc` lookup; `ignore_index=True` prevents that.

(3) With `a` indexed [0, 1] and `b` indexed [1, 2], `pd.concat([a, b], axis=1)` has three rows, 0, 1, 2. Only label 1 is in both, so only that row is filled on both sides; row 0 is NaN in `b`’s column and row 2 is NaN in `a`’s column, because `axis=1` aligns on the index label, not on position.

6. Reshaping: Wide and Long

The same data can be laid out in two ways, and many tasks are easy in one layout and awkward in the other. Being able to switch between them is a quietly powerful skill.

Definition 10 (Wide and long format). In **wide** format, each variable has its own column and each entity is one row (a spreadsheet-style table). In **long** format, there is one row per **observation**, with one column naming the variable and another holding its value. The same facts, arranged differently.

Example — The same revenue, two shapes. Wide — one column per quarter, easy for a person to read across:

region	Q1	Q2
North	1200	1300
South	1500	1400

Long — one row per (region, quarter) observation, easy for a computer to group and plot:

region	quarter	revenue
North	Q1	1200
South	Q1	1500
North	Q2	1300
South	Q2	1400

Both hold the same information. The wide form is better for reading and for a report; the long form is what groupby, merging, and most plotting libraries expect. You will move between them often.

Definition 11 (pivot and melt).

- **pivot** turns **long into wide**: it spreads the values of one column out into new columns.
- **melt** turns **wide into long**: it gathers a set of columns into two, one naming the old column and one holding its value.

Example — Reshaping the table, and back again. Build the wide table in code, then reshape it. `melt` gathers the two quarter columns into a single quarter column paired with a revenue column:

```
wide = pd.DataFrame({
    "region": ["North", "South"],
    "Q1": [1200, 1500],
    "Q2": [1300, 1400],
})
long = wide.melt(id_vars="region", var_name="quarter", value_name="revenue")
```

```

      region quarter  revenue
0   North     Q1     1200
1   South     Q1     1500
2   North     Q2     1300
3   South     Q2     1400

```

`id_vars="region"` names the column to keep as an identifier; every other column (Q1, Q2) is gathered. `var_name` and `value_name` name the two columns created: one holding the old column name, one holding its value.

`pivot` is the inverse. It spreads the quarter values back out into columns:

```
back = long.pivot(index="region", columns="quarter", values="revenue").reset_index()
```

```

quarter region    Q1    Q2
0         North  1200  1300
1         South  1500  1400

```

`back` is the original wide table again — `pivot` and `melt` undo each other. `pivot` makes the index column (`region`) the row index, so `reset_index` turns it back into an ordinary column.

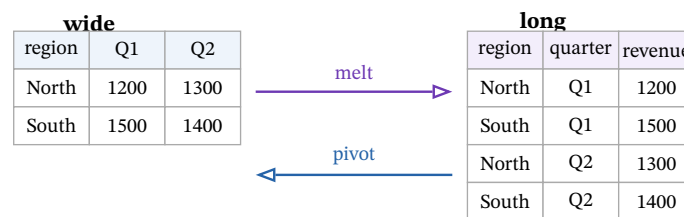


Figure 5: `melt` gathers the wide quarter columns into a single quarter column with a paired revenue value; `pivot` spreads them back out. The information is unchanged; only its shape is.

Pitfall. `pivot` fails if a single (`index`, `columns`) pair appears more than once, because it does not know which value to keep, and it raises an error. When duplicates are expected (several sales on the same quarter in the same region), use `pivot_table` instead, which **aggregates** the collisions (for example, sums them). Reach for `pivot` only when each combination is unique.

Exercises

6.1. Build the small wide table of revenue by region and quarter shown in this chapter. Reshape it to long with `melt`, then back to wide with `pivot`, and confirm you recover the original. State one task that is easier in each layout.

Answers. (1) `long = wide.melt(id_vars="region", var_name="quarter", value_name="revenue"); back = long.pivot(index="region", columns="quarter", values="revenue").reset_index()`. `back` matches the original wide table. Reading one region's two quarters side by side is easier in wide format; grouping or plotting revenue by quarter is easier in long format.

7. Applying Functions

Pandas' built-in operations cover most needs, but sometimes you must run **your own** logic on every value, column, or row. `apply` is the general tool for that. It pairs naturally with a **lambda**: a small function with no name, written on one line, of the form `lambda x: <expression using x>`.

Example — A custom calculation, value by value. Suppose you want each shop's revenue in cents rather than in whole currency units. There is no built-in “to cents”, so you apply a function of your own:

```
df["cents"] = df["revenue"].apply(lambda x: round(x * 100))
```

Pandas calls the little function once per value of revenue, with `x` set to that value, and collects the results into a new column. The lambda is just a one-line function: `lambda x: round(x * 100)` means “given `x`, return `round(x * 100)`”.

7.1. Series `apply` and DataFrame `apply`

Two different objects can call `apply`, and they behave differently. Keeping them apart is the whole of this section.

A Series calls `apply` element by element. The function receives one value at a time; there is nothing to configure, and `Series.apply` takes no `axis` argument at all. That is the call from the example above:

```
df["orders"].apply(lambda x: x + 1)      # one value at a time; no axis here
```

A DataFrame calls `apply` on a whole Series at a time, and the `axis` argument chooses **which** Series. It does not switch between “one value” and “one row”; it switches between whole columns and whole rows:

- `axis=0`, the default, hands the function each **column** (a Series);
- `axis=1` hands the function each **row** (a Series).

Either way the function is handed an entire Series, never a single value — the part beginners most often get wrong.

Example — The default axis runs down the columns. With the default `axis=0`, `apply` calls the function once per **column**, passing the whole column. Here the function returns each column's range, its maximum minus its minimum:

```
df[["revenue", "orders"]].apply(lambda col: col.max() - col.min())
```

```
revenue    2050
orders      220
dtype: int64
```

The function ran twice, once per column: revenue gives $2000 - (-50) = 2050$ and orders gives $300 - 80 = 220$. It was never called on a single value.

Pass `axis=1` and the same method hands over a whole **row** instead, so the function can combine several columns of one shop:

```
df.apply(lambda row: row["revenue"] / row["orders"], axis=1) # a whole row at a time
```

Each row arrives as a Series; the function divides that shop's revenue by its orders to give its revenue per order.

Pitfall. Do not expect `df.apply(f)` to call `f` on every individual value. With the default `axis=0` it calls `f` once per **column**, handing over the entire column, so `df.apply(lambda x: x + 1)` passes whole columns to `x`, not numbers. To run a function on each value of one column, call it on that column, `df["orders"].apply(f)`, where `apply` is element-wise and no `axis` exists. To run it across each row, pass `axis=1`.

Pitfall. `apply` is flexible but **slow**: it runs your Python function once per row, which on a million rows means a million function calls. Whenever a built-in **vectorised** operation exists, prefer it. `df["revenue"] / df["orders"]` is both shorter and much faster than `df.apply(lambda row: row["revenue"] / row["orders"], axis=1)`, and gives the same result. Reach for `apply` only for logic that pandas' own operations cannot express.

Remark. A `lambda` is convenient but has no name. If the logic grows beyond one line, or you want to reuse it, write a named `def` function and pass it by name, as in `df["revenue"].apply(to_cents)`. `apply` accepts any function, whether a `lambda` or not.

Exercises

7.1. Add a `revenue_per_order` column, first with `apply` and a `lambda`, then as a vectorised operation. Explain why the vectorised version is preferable.

Answers. (1) `df["revenue_per_order"] = df.apply(lambda r: r["revenue"] / r["orders"], axis=1)` and `df["revenue_per_order"] = df["revenue"] / df["orders"]` give the same column. The vectorised version is shorter and much faster, because it does not call a Python function once per row.

8. The End-to-End Pipeline

Every technique so far is a single move; real work **chains** them together, from a raw file to a clean, summarised result. This chapter puts the whole path in one script.

Definition 12 (The four stages).

1. **Load** the raw file into a DataFrame.
2. **Inspect** it — shape, types, missing values — to learn what you are dealing with.
3. **Clean** it — handle missing values and clear errors, recording each decision.
4. **Summarise** it — group and aggregate into the table that answers the question.

Example — Raw shops file to per-region summary.

```
import pandas as pd

# 1. Load
df = pd.read_csv("shops.csv")

# 2. Inspect
print(df.shape)           # how big?
print(df.info())          # types, and where values are missing
print(df.describe())      # spread; check min and max for errors

# 3. Clean
df["region"] = df["region"].fillna("Unknown") # keep the shop, flag the gap
df = df[df["revenue"] >= 0]                  # drop the impossible negative
revenue

# 4. Summarise
summary = (
    df.groupby("region")["revenue"]
      .agg(["sum", "mean", "count"])
      .reset_index()
      .sort_values("sum", ascending=False)
)
print(summary)
```

The result:

	region	sum	mean	count
1	North	4180	1045.000000	4
4	West	3570	1190.000000	3
2	South	3400	1133.333333	3
0	East	2800	1400.000000	2
3	Unknown	540	540.000000	1

Read the cleaning block as a set of **decisions you can defend**, not as mechanical steps: the shop with no region is kept but flagged as "Unknown", so it still counts toward the total; the row with negative revenue is removed because a negative revenue is impossible. (Notice that removing it changed the East group from 3 shops to 2, and raised its total to 2800.) Each line is a judgement

you could explain to someone who doubts it, which is exactly the standard the reporting chapter (Chapter 10) holds you to.

Remark. Notice the parentheses around the summary block. Wrapping a chain of calls in (...) lets you put each step (`.groupby`, `.agg`, `.reset_index`, `.sort_values`) on its own line. The chain then reads top to bottom like a recipe, and you can comment out any single step. This is easier to read than one long line or a pile of intermediate variables.

Pitfall. Order matters in a pipeline. If you `groupby("region")` **before** filling the missing region, that shop vanishes from the totals (recall that `groupby` drops NaN keys); if you compute a mean **before** removing the impossible value, the `-50` pulls it down. Inspect first, clean second, summarise last, and within cleaning, fix the errors that would distort a later step before you take that step.

Exercises

8.1. Write a complete pipeline on `shops.csv`: load the file, inspect it (shape, types, missing values), clean it (recording each decision as a comment), and summarise revenue by region with a grouped aggregation. Keep the load → inspect → clean → summarise order.

Answers. (1) The pipeline in this chapter is a model answer: load with `read_csv`; inspect with `shape`, `info`, and `describe`; clean by filling the missing region with "Unknown" and dropping the row whose revenue is negative; then `groupby("region")["revenue"].agg(["sum", "mean", "count"])`. Order matters: filling the region before grouping keeps that shop in the totals, and dropping the `-50` row before averaging keeps it out of the East mean.

9. Visualization

A summary table answers a question in numbers; a **chart** answers it in a shape the eye reads at a glance. A column of fourteen revenues is hard to compare in your head, but as bars of different heights the largest and smallest jump out at once. This chapter covers the four charts that carry most of everyday analysis, and — just as important — **which** chart answers **which** kind of question.

Two libraries do the drawing. **matplotlib** is the foundation: it can draw anything, one instruction at a time. **seaborn** is a thin, friendlier layer built on top of it that turns a DataFrame straight into a good-looking chart in one call. We use seaborn for the chart itself and matplotlib's `plt` for the finishing touches — the title, the axis labels, and showing or saving the figure. Import both, once, at the top of your script:

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid") # a clean default look for every chart
```

Two matplotlib calls end every chart. `plt.show()` opens it in a window; `plt.savefig("chart.png")` writes it to a file to put in a report. Throughout this chapter, `df` is the **cleaned** shops table from Chapter 8 (the missing region filled with "Unknown", the impossible negative revenue dropped).

9.1. Choosing the right chart

The chart is chosen by the **question**, not by taste. Four questions cover most of what a first analysis asks, and each has a natural chart.

Framework 1 (Question to chart).

- **Compare one value across categories** (“which region earns most?”) → a **bar chart**: one bar per category, height for the value.
- **See how one numeric variable is distributed** (“are most shops small or large?”) → a **histogram**: values bucketed into ranges, bar height for how many fall in each.
- **Follow a value along an ordered axis, usually time** (“how did revenue move month by month?”) → a **line chart**: points joined in order.
- **See whether two numeric variables are related** (“do busier shops earn more?”) → a **scatter plot**: one dot per row, placed by its two values.

9.2. Bar chart: comparing across categories

A bar chart puts one bar per category side by side, so their heights can be compared directly. Feed seaborn a small table with one row per category — here, total revenue per region from a groupby — and name the category column and the value column:

```
by_region = df.groupby("region")["revenue"].sum().reset_index()
plt.figure(figsize=(6, 3.5))
sns.barplot(data=by_region, x="region", y="revenue")
```

```
plt.title("Total revenue by region")
plt.ylabel("total revenue")
plt.show()
```

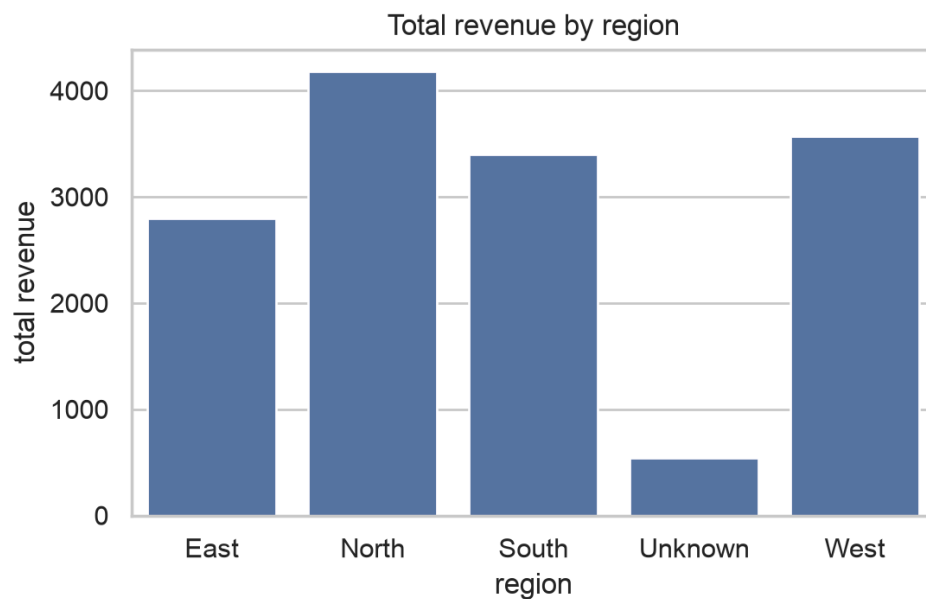


Figure 6: A bar chart compares one value across categories. North’s total revenue is the tallest bar; the single “Unknown”-region shop is the shortest. Heights are read off the shared vertical axis.

Figure 6 answers “which region earns most in total?” instantly: North leads, the “Unknown” bar is tiny because it is a single shop. The eye compares heights far faster than it compares a column of numbers.

9.3. Histogram: the shape of one variable

A histogram is **not** a bar chart, though both draw bars. A bar chart compares given categories; a histogram takes **one numeric column**, chops its range into equal buckets (bins), and draws how many values fall in each — showing the variable’s **distribution**. There is one input column, not two:

```
plt.figure(figsize=(6, 3.5))
sns.histplot(data=df, x="orders", bins=6)
plt.title("Distribution of orders")
plt.show()
```

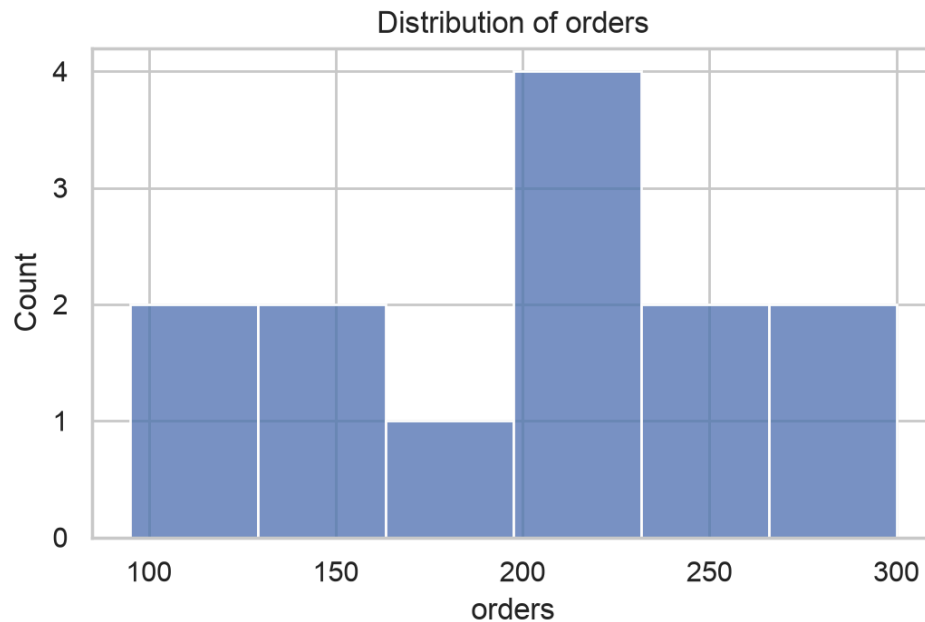


Figure 7: A histogram shows how one numeric variable is distributed. Each bar counts the shops whose orders fall in that range; the tallest band, around 200–230 orders, holds four shops.

Figure 7 answers “how busy is a typical shop?” Orders run from under 100 to 300, and the fullest band sits around 200–230, where four shops fall. The `bins` argument sets how many buckets to use: too few hides the shape, too many makes it jagged, so try a couple of values.

Pitfall. A bar chart and a histogram look alike but ask different questions. A bar chart’s x-axis lists **categories you chose** (regions, products), one bar each; a histogram’s x-axis is a **numeric range cut into bins**, and its bars must touch, because they cover a continuous scale. If your x-axis is a number line, you want a histogram; if it is a list of names, you want a bar chart.

9.4. Line chart: a value along an ordered axis

A line chart joins points in the order of the x-axis, so it fits a value measured along something ordered — most often time. Joining the dots draws the eye along the trend. This one uses a small monthly table and plain matplotlib:

```
months = pd.DataFrame({
    "month":  ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "revenue": [10200, 10800, 9900, 11500, 12100, 12800],
})
plt.figure(figsize=(6, 3.5))
plt.plot(months["month"], months["revenue"], marker="o")
plt.title("Total revenue by month")
plt.xlabel("month")
plt.ylabel("total revenue")
plt.show()
```

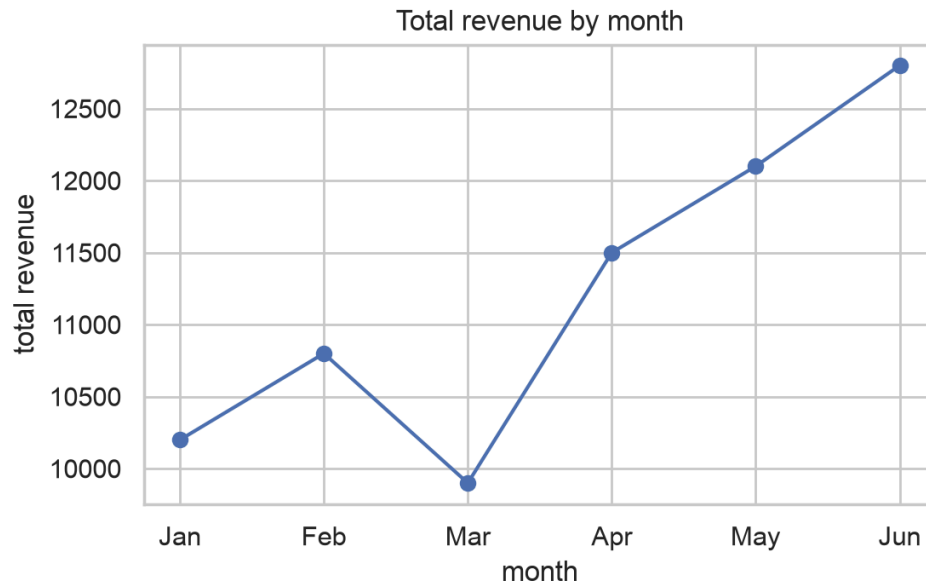


Figure 8: A line chart follows a value along an ordered axis. Revenue climbs from January to June, apart from a dip in March; the joined line makes the trend and the one interruption easy to see.

Figure 8 answers “which way is revenue heading?”: upward overall, with a single dip in March. The line is only honest when the x-axis has a real order — months, days, years. Joining unordered categories with a line would draw a trend that does not exist, which is the next pitfall.

Pitfall. Use a line chart **only** when the x-axis is ordered. Joining categories that have no natural order — regions, product names — with a line invents a slope between them that means nothing: the picture would change if you listed the categories in another order. For unordered categories, use a bar chart.

9.5. Scatter plot: relating two variables

A scatter plot places one dot per row, using two numeric columns as the two coordinates. It answers whether the two move together. Give seaborn the two columns:

```
plt.figure(figsize=(6, 3.5))
sns.scatterplot(data=df, x="orders", y="revenue")
plt.title("Revenue against orders")
plt.show()
```

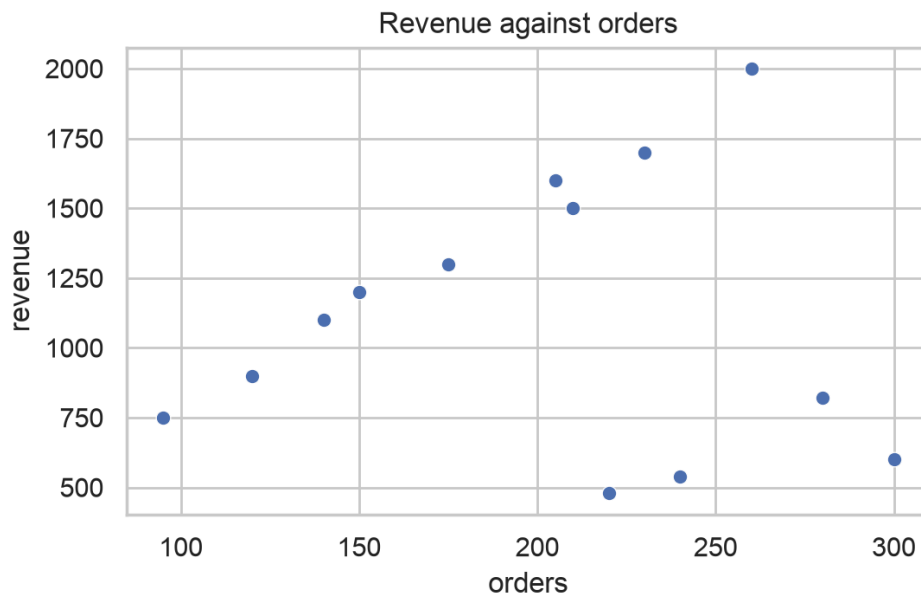


Figure 9: A scatter plot shows the relationship between two numeric variables. Each dot is one shop, placed by its orders and its revenue. More orders tends to go with more revenue, but loosely: several busy shops still earn little.

Figure 9 answers “do busier shops earn more?” The cloud drifts upward — more orders tends to mean more revenue — but the link is loose: a few shops take many orders yet earn little, so orders alone do not fix revenue. A scatter plot shows both that a relationship exists **and** how tight it is.

Pitfall. A rising scatter shows the two variables **move together**; it does not show that one **causes** the other. Busier shops might earn more for many reasons, or a third factor (shop size, say) might drive both. Reading a cause into a scatter is the same over-claim the reporting chapter (Chapter 10) warns against — the plot describes, it does not explain.

Exercises

9.1. On the cleaned shops table, draw a **bar chart** of **mean rating per category**. Which category is rated highest? State the question the chart answers in one sentence.

9.2. Draw a **histogram** of the revenue column with bins=5, then again with bins=10. Describe how the picture changes, and say what a histogram tells you that a bar chart of the same column could not.

9.3. Using the small monthly table from this chapter, draw a **line chart** of revenue by month. Explain why a line chart suits this data but would be wrong for a chart of revenue by region.

9.4. Draw a **scatter plot** of rating (x) against revenue (y). Say whether the two appear related, and write one sentence that stops short of claiming a cause.

9.5. For each question below, name the single most appropriate chart and say why: (a) “Which product category has the most shops?” (b) “Are ratings mostly high, or spread out?” (c) “Does a shop’s number of orders track its revenue?” (d) “How did total revenue change over the last six months?”

Answers. (1) `cat = df.groupby("category")["rating"].mean().reset_index()`, then `sns.barplot(data=cat, x="category", y="rating")`. A bar chart suits a value compared across chosen categories; it answers “which category is rated highest on average?”

(2) `sns.histplot(data=df, x="revenue", bins=5)` and again with `bins=10`. More bins show finer detail but a rougher, more jagged shape; fewer bins are smoother but hide structure. A histogram shows the **distribution** of one column — where values cluster and how spread they are — which a bar chart, comparing separate categories, cannot.

(3) `plt.plot(months["month"], months["revenue"], marker="o")`. Months have a natural order, so joining them traces a real trend over time. Regions have no order, so a line between them would imply a slope that changes with the listing order and means nothing; revenue by region belongs in a bar chart.

(4) `sns.scatterplot(data=df, x="rating", y="revenue")`. The dots show little or no clear trend, so rating and revenue appear at most weakly related. A safe sentence: "Higher-rated shops do not clearly earn more in this data." No cause is claimed.

(5) (a) bar chart — a count compared across chosen categories. (b) histogram — the distribution of one numeric column. (c) scatter plot — the relationship between two numeric columns. (d) line chart — a value along an ordered time axis.

10. Descriptive Reporting

A summary table is not the finish line. A manager does not want a DataFrame; they want to know **what it means**. The final, and often undervalued, skill of this course is turning a table of numbers into a short paragraph a person can act on — usually beside a chart (Chapter 9) that makes the same point at a glance.

Definition 13 (A descriptive report). A descriptive report states, in plain language, what the summary shows: which groups are largest and smallest, where the values concentrate, and any notable gaps or outliers. It **describes what the data says** and does not claim **why**. Causes lie beyond what a descriptive summary can support.

Example — From table to paragraph. Given the per-region summary from the previous chapter (North: sum 4180, mean 1045, 4 shops; West: 3570, 1190, 3; South: 3400, 1133, 3; East: 2800, 1400, 2; Unknown: 540, 540, 1), a good report reads:

“North has the highest total revenue (4180), mainly because it has the most shops (4) rather than the highest average. West and South follow with similar totals (3570 and 3400). East’s average revenue per shop is the highest of all (1400), but this rests on only two shops, so it is a weak basis for a conclusion. One shop has no region recorded and appears as ‘Unknown’; it is a single shop, but a missing region should be investigated in case it points to a data-collection problem.”

Every sentence points back to a number in the table, compares groups, and stops short of inventing a cause. That is the genre.

Pitfall. The main error in reporting is sliding from description to unfounded **cause**: “North earns more **because** its customers are wealthier” is not in the data — you measured revenue, not wealth. State what the numbers show, and flag what would need more data to explain. Confident over-claiming from a descriptive table is how an analysis loses its credibility.

Pitfall. The opposite error is burying the finding under every number. A report is not the table read aloud; it is the **two or three things that matter** — the biggest group, the surprising contrast, the worrying gap — stated plainly. If your paragraph mentions every cell, it has interpreted none of them.

Remark. Everything in this course was practised on one small, tidy table so that you could check every result by eye. A real dataset is larger and messier — many more columns, dirtier types, gaps that are harder to judge — but the path does not change: load, inspect, clean with stated reasons, summarise, visualise, and report what the numbers show. What grows is the care each step needs, not the number of steps. The last exercise below asks you to walk that path once on a dataset of your own.

Exercises

10.1. Take the per-region summary produced by the pipeline in the previous chapter and write a short paragraph interpreting it: name the leading region by total and by average, flag any group whose figure rests on very few shops, and note the "Unknown" region as a data-quality point. Claim no cause.

10.2. Find a real dataset of at least a few hundred rows and several columns — many governments and organisations publish free CSV files on open-data portals — and run the full pipeline on it: load, inspect, clean (justifying each decision), and summarise along one grouping column. Draw at least one chart that fits the question (Chapter 9), and finish with a short paragraph interpreting the summary and its chart, in the same descriptive genre, without claiming a cause.

Answers. (1) A good paragraph resembles the one in this chapter's example: North leads on total revenue (4180), mainly through having the most shops; East has the highest average (1400) but on only two shops, so that figure is a weak basis for a conclusion; the single "Unknown" region is one shop but should be investigated. Every sentence points to a number, and none claims why.

(2) No single answer, but a good one loads the file, reports its shape and dtypes, checks info and describe for missing values and impossible figures, cleans them with a stated justification, and ends with a two- or three-sentence paragraph naming the largest and smallest groups and any striking gap — describing what the numbers show, not why. The method is exactly the one practised on `shops.csv`; only the size and messiness of the data change.

Remark (Source). The split-apply-combine strategy that `groupby` carries out (Chapter 4) was named and set out by Hadley Wickham in "The Split-Apply-Combine Strategy for Data Analysis", *Journal of Statistical Software* 40(1), 2011.